

NumPy - kompakte Lerneinheit

Grundlagen, Beispiele, Übungen und Lösungen für den praktischen Einstieg in numerisches Python

Lernziel	Nach dieser Einheit kannst du Arrays erzeugen, auswählen, verändern, berechnen und auswerten.
Voraussetzung	Grundlegende Python-Kenntnisse: Variablen, Listen, Schleifen und Funktionen.
Typische Dauer	Ca. 60-90 Minuten inklusive Übungen.

Warum NumPy? NumPy ist die zentrale Bibliothek für numerisches Rechnen in Python. Statt viele Werte einzeln in Schleifen zu bearbeiten, arbeitet man mit Arrays. Das ist meist kürzer, übersichtlicher und deutlich schneller.

```
import numpy as np
```

1. Das NumPy-Array

Das wichtigste Objekt in NumPy ist das **ndarray**. Es ähnelt einer Python-Liste, ist aber für numerische Berechnungen optimiert. In einem Array haben die Elemente normalerweise denselben Datentyp.

Array aus einer Python-Liste:

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a)
print(type(a))
```

Mehrdimensionale Arrays:

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix)
```

Wichtige Eigenschaften

Attribut	Bedeutung	Beispiel
shape	Form des Arrays	(2, 3)
ndim	Anzahl Dimensionen	2
size	Anzahl aller Elemente	6
dtype	Datentyp der Elemente	int64 / int32

```
print(matrix.shape) # (2, 3)
print(matrix.ndim)  # 2
print(matrix.size)  # 6
print(matrix.dtype)
```

2. Arrays erzeugen

Für viele Aufgaben musst du Werte nicht von Hand eintippen. NumPy kann Arrays gezielt erzeugen.

```
np.zeros(5)          # [0. 0. 0. 0. 0.]
np.ones(4)           # [1. 1. 1. 1.]
np.arange(0, 10, 2)   # [0 2 4 6 8]
np.linspace(0, 1, 5)  # 5 gleichmäßig verteilte Werte
```

Zufallszahlen:

```
rng = np.random.default_rng(42)

print(rng.integers(1, 11, size=5))
print(rng.random(5))
```

3. Indexing und Slicing

Wie bei Listen beginnt die Indizierung bei 0. Mit Slices kannst du mehrere Werte auf einmal auswählen.

```
a = np.array([10, 20, 30, 40, 50])

print(a[0])      # 10
print(a[-1])     # 50
print(a[1:4])    # [20 30 40]
print(a[::-2])   # [10 30 50]
```

Bei einer Matrix verwendest du **Zeile, Spalte**:

```
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(m[1, 2])   # 6
print(m[0, :])   # erste Zeile
print(m[:, 1])   # zweite Spalte
```

Boolean Indexing ist besonders wichtig: Du filterst Werte anhand einer Bedingung.

```
a = np.array([3, 8, 12, 5, 20])

print(a[a > 10]) # [12 20]
```

4. Rechnen mit Arrays - ohne Schleifen

NumPy führt Rechenoperationen elementweise aus. Das nennt man oft **Vektorisierung**.

```
a = np.array([1, 2, 3, 4])

print(a + 10)
print(a * 2)
print(a ** 2)
print(np.sqrt(a))
```

Auch zwei gleich große Arrays können direkt miteinander verrechnet werden:

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)    # [11 22 33]
print(a * b)    # [10 40 90]
```

Das ersetzt häufig Python-Schleifen wie for x in liste.

5. Aggregationen

Aggregationen reduzieren viele Werte auf Kennzahlen.

```
a = np.array([4, 8, 15, 16, 23, 42])

print(a.sum())
print(a.mean())
print(a.min())
print(a.max())
print(a.std())
```

Bei Matrizen kann die Berechnung entlang einer Achse erfolgen:

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(m.sum(axis=0))    # spaltenweise: [5 7 9]
print(m.sum(axis=1))    # zeilenweise: [6 15]
```

6. Shape, Reshape und Flatten

Die Form eines Arrays wird über shape beschrieben. Mit reshape kannst du dieselben Werte anders anordnen.

```
a = np.arange(1, 13)
print(a.shape)

m = a.reshape(3, 4)
print(m)
print(m.shape)
```

Die Anzahl der Elemente muss gleich bleiben: 12 Werte können z. B. als (3,4), (4,3) oder (2,6) angeordnet werden.

```
flat = m.flatten()
print(flat)
```

7. Broadcasting

Broadcasting bedeutet, dass NumPy kleinere Strukturen automatisch passend auf größere Arrays anwendet.

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(m + 10)
```

Auch ein passender Vektor kann auf jede Zeile angewendet werden:

```
offset = np.array([100, 200, 300])
print(m + offset)
```

NumPy vergleicht Shapes von rechts nach links. Dimensionen müssen gleich sein oder eine der Dimensionen muss 1 sein.

8. Praktisches Mini-Beispiel

Angenommen, du hast Messwerte von Temperaturen:

```
temps = np.array([18.5, 20.1, 21.8, 19.7, 23.4, 24.0])

print("Mittelwert:", temps.mean())
print("Maximum:", temps.max())
print("Über 22 Grad:", temps[temps > 22])

# Celsius -> Fahrenheit
fahrenheit = temps * 9 / 5 + 32
print(fahrenheit)
```

Das Beispiel zeigt den typischen NumPy-Stil: **Daten als Array halten, Operationen auf das gesamte Array anwenden, Ergebnisse filtern oder aggregieren.**

9. Übungen

Übung 1 - Array erzeugen

Erzeuge ein Array mit den Zahlen 1 bis 10. Gib anschließend das dritte Element aus.

Übung 2 - Filtern

Gegeben ist `a = np.array([4, 11, 7, 20, 3, 15])`. Gib nur Werte größer als 10 aus.

Übung 3 - Statistik

Berechne für `[12, 15, 14, 10, 19]` Summe, Mittelwert, Minimum und Maximum.

Übung 4 - Reshape

Erzeuge die Zahlen 1 bis 12 und forme sie in eine Matrix mit 3 Zeilen und 4 Spalten um.

Übung 5 - Vektorisierung

Gegeben sind Preise `[10, 20, 30, 40]`. Erhöhe alle Preise gleichzeitig um 19 %.

Übung 6 - Matrix

Erzeuge eine 3x3-Matrix mit den Zahlen 1 bis 9. Gib die zweite Zeile und die dritte Spalte aus.

Bonus

Erzeuge 100 Zufallszahlen zwischen 1 und 100 und bestimme Mittelwert, Minimum, Maximum sowie die Anzahl der Werte größer als 50.

10. Lösungen

Übung 1

```
a = np.arange(1, 11)
print(a[2])
```

Übung 2

```
a = np.array([4, 11, 7, 20, 3, 15])
print(a[a > 10])
```

Übung 3

```
a = np.array([12, 15, 14, 10, 19])
print(a.sum())
print(a.mean())
print(a.min())
print(a.max())
```

Übung 4

```
m = np.arange(1, 13).reshape(3, 4)
print(m)
```

Übung 5

```
preise = np.array([10, 20, 30, 40], dtype=float)
neue_preise = preise * 1.19
print(neue_preise)
```

Übung 6

```
m = np.arange(1, 10).reshape(3, 3)
print(m[1, :])
print(m[:, 2])
```

Bonus

```
rng = np.random.default_rng()
a = rng.integers(1, 101, size=100)

print(a.mean())
print(a.min())
print(a.max())
print(np.sum(a > 50))
```

11. NumPy-Spickzettel

Aufgabe	Syntax
Import	<code>import numpy as np</code>
Array	<code>np.array([1, 2, 3])</code>
0 / 1	<code>np.zeros(5), np.ones(5)</code>
Zahlenfolge	<code>np.arange(start, stop, step)</code>
Gleichmäßige Werte	<code>np.linspace(start, stop, n)</code>
Form	<code>a.shape</code>
Dimensionen	<code>a.ndim</code>
Elementanzahl	<code>a.size</code>
Auswahl	<code>a[0], a[1:4], a[a > 5]</code>
Rechnen	<code>a + 2, a * 3, a ** 2</code>
Statistik	<code>a.sum(), mean(), min(), max(), std()</code>
Umformen	<code>a.reshape(...), a.flatten()</code>
Matrixauswahl	<code>m[zeile, spalte]</code>
Zufall	<code>np.random.default_rng()</code>

Merksatz: Wenn du viele numerische Werte gleichzeitig bearbeiten willst, denke zuerst in Arrays statt in einzelnen Variablen oder Schleifen.